Observer is a behavioral design pattern that lets you define a subscription mechanism to notify multiple objects about any events that happen to the object they're observing.

Think of a YouTube Channel as the Subject and Subscribers as the Observers. When the channel uploads a new video, it doesn't call every subscriber individually. Instead, it sends out a single notification, and everyone who "subscribed" to that channel receives the update automatically.

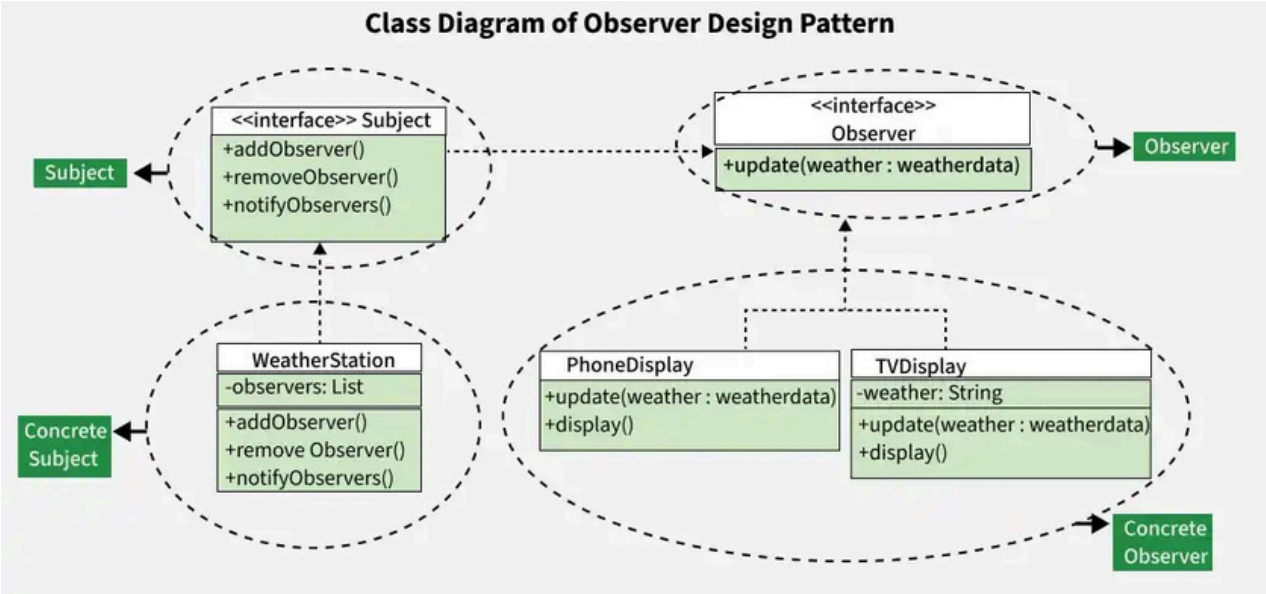
```java
import java.util.ArrayList;
import java.util.List;

// Subject Interface
interface Subject {
    void subscribe(Observer observer);
    void unsubscribe(Observer observer);
    void notifyObservers(String videoTitle);
}

// Observer Interface
interface Observer {
    void update(String videoTitle);
}

// Concrete Subject
class YouTubeChannel implements Subject {
    private List<Observer> subscribers = new ArrayList<>();

    @Override
    public void subscribe(Observer observer) {
        subscribers.add(observer);
    }

    @Override
    public void unsubscribe(Observer observer) {
```

```java
    @Override
    public void unsubscribe(Observer observer) {
        subscribers.remove(observer);
    }

    @Override
    public void notifyObservers(String videoTitle) {
        for (Observer sub : subscribers) {
            sub.update(videoTitle);
        }
    }

    public void uploadVideo(String title) {
        System.out.println("Channel uploaded: " + title);
        notifyObservers(title);
    }
}

// Concrete Observer
class Subscriber implements Observer {
    private String name;

    public Subscriber(String name) {
        this.name = name;
    }

    @Override
    public void update(String videoTitle) {
```

```java
    @Override
    public void update(String videoTitle) {
        System.out.println("Hey " + name + ", new video notification: " + videoT
    }
}

// Test Class
public class ObserverPatternTest {
    public static void main(String[] args) {
        YouTubeChannel techChannel = new YouTubeChannel();

        Observer user1 = new Subscriber("Alice");
        Observer user2 = new Subscriber("Bob");

        techChannel.subscribe(user1);
        techChannel.subscribe(user2);

        // Notify all subscribers
        techChannel.uploadVideo("Java Design Patterns Tutorial");

        // Unsubscribe one user
        techChannel.unsubscribe(user1);

        // Notify remaining subscribers
        techChannel.uploadVideo("Advanced Spring Boot");
    }
}
```

Why this is an Observer Pattern

- **One-to-Many Relationship:** One `YouTubeChannel` maintains a list of multiple `Subscriber` objects.
- **Loose Coupling:** The `YouTubeChannel` doesn't need to know the specific details of the `Subscriber` class; it only knows that they implement the `Observer` interface.
- **Automatic Triggering:** The notification process is triggered automatically by the state change (the `uploadVideo` method), ensuring the Subject doesn't have to manually manage each update.
- **Dynamic Subscription:** Observers can be added (`subscribe`) or removed (`unsubscribe`) at runtime without changing the Subject's code.

3. Structural Comparison

Feature	Mediator Pattern	Observer Pattern
Problem	Tight coupling between many objects.	Need to update others when one state changes.
Control	**Centralized**: The Mediator contains the logic.	**Distributed**: The Subject/ Observers handle logic.
Coupling	Objects are decoupled from each other but coupled to the Mediator.	The Subject and Observers are loosely coupled.
Analogy	**Airport Tower**: Coordinates all planes.	**Newsletter**: Subscribing to updates.